

Kiosk Smart System

Occupancy Based Smart System

Smart Air Monitoring System

Report submitted in partial fulfillment for the degree of
B.Tech in Electronics & Computer Science

Submitted By
SAGNIK PAL



School of Electronics
KIIT University
Bhubaneswar, Odisha, India
Under the Supervision of
Mr Tarun Lakhmani
IFB Industries Pvt Ltd, Goa

DECLARATION

I hereby declare that the dissertation report entitled “ **KIOSK BASED SMART SYSTEM** ”, “ **OCCUPANCY BASED SMART SYSTEM** ” and “ **SMART AIR MONITORING SYSTEM** ” submitted by me, for the partial fulfillment of the degree of B. Tech in Electronics and Computer Science, KIIT University is a record of bona fide work carried by me under the supervision and guidance of Mr Tarun Lakhman , IFB Industries Pvt Ltd , Goa.



Sagnik Pal

ABSTRACT

This thesis presents a comprehensive exploration of real-time, embedded **Internet of Things (IoT) systems**, developed as part of a three-month industrial internship. It focuses on the design, implementation, and integration of **three modular and scalable IoT projects** tailored for **smart home automation, environmental sensing, and intelligent control systems**. Each project demonstrates a full-stack approach, encompassing **sensor interfacing, microcontroller programming, network communication protocols, data visualization, and user interface design**.

The first project, **Kiosk Smart System**, introduces a **centralized touchscreen appliance control platform** built on **Raspberry Pi**. Featuring a **21-inch kiosk interface**, it allows real-time operation of smart appliances such as **air conditioners, washing machines, fans, lights, and refrigerators**. Developed using **HTML, CSS, JavaScript, MQTT, and Node-RED**, the system offers a **modular UI, MQTT-based messaging architecture, and a scalable backend**, positioning it as a foundational tool for smart home orchestration.

The second project, **Occupancy Sensor Automation System**, leverages **environmental awareness for energy-efficient automation**. Utilizing a **PIR motion sensor and a CO₂ sensor (SCD40)**, the system detects room occupancy and controls appliances using **GPIO-based relay switching**. It

integrates **Python scripting, Node-RED flows, and CSV-based data logging** to automate real-time decision-making. A **Node-RED dashboard** offers live sensor visualization.

The third project, **Smart Air Monitoring System**, delivers a **real-time air quality monitoring dashboard** powered by **multi-sensor fusion**. Sensors such as **DHT22, PMS5003, MQ135, and SCD40** capture parameters including **temperature, humidity, CO₂ levels, PM2.5, PM10, and AQI**. The system uses **MQTT for data transmission** and **WebSocket protocols** to stream data dynamically to a custom **HTML/CSS/JavaScript web interface**.

Collectively, these projects reflect core competencies in **embedded system design, IoT architecture, real-time data communication, and full-stack development**. They demonstrate the feasibility of building **cost-effective, open-source, and integrated IoT solutions** for modern applications.

The work also highlights future potential for **cloud integration (Firebase, AWS IoT), mobile app control, AI/ML-based predictions, and cross-system interoperability**. It contributes meaningfully to the field of **IoT-driven smart automation**, emphasizing **practical implementation, academic relevance, and industrial applicability** in the evolving landscape of **connected technologies**.

ACKNOWLEDGEMENT

The completion of these projects could not have been successful without the expert guidance of **Mr. Tarun Lakhmani** and **Mr Bhavesh Naik**. I would also like to extend my heartfelt gratitude to Mr. Bhavesh Naik for the much needed help and directions throughout the completion of my thesis.

I would like to acknowledge my fellow lab members of Dr. Akshaya Kumar Pati sir who was very helpful and resourceful throughout. A lot of gratitude to the Human Resource of IFB Industries, Goa, for giving me this opportunity.

Also, I am grateful to Dr. Suprava Patnaik, Dean of KIIT School of Electronics and the faculty members of KIIT School of Electronics for their constant support above and beyond this project.



Sagnik Pal

TABLE OF CONTENT

CHAPTER 1

INTRODUCTION

Page No

1.1. IoT in Modern Applications	9
1.2. Motivation and Scope of the Thesis	10
1.2.1. Smart Home Appliance Control System	11
1.2.2. Occupancy-Based Automation System	12
1.2.3. Real-Time Smart Air Quality Monitoring System	13
1.3. Objective of the Thesis	14

CHAPTER 2

LITERATURE REVIEW

2.1. Communication Protocols for IoT Systems	15
2.2. Sensor-Based Automation Systems	16
2.3. Real-Time Environmental Monitoring	17
2.4. Relevance of Integrated IoT Projects	18

CHAPTER 3

PROJECT 1: KIOSK SMART SYSTEM

3.1. Abstract	19
3.2. Methodology	20
Phase 1: Initialization & Environment Setup	
Phase 2: Frontend Development	
Phase 3: Backend Integration	
Phase 4: Hosting and Remote Access	
Phase 5: Final Testing and Validation	
3.3. System Architecture	23
3.4. Hardware and Software Design	24
3.5. Results and Discussion	25
3.6. Conclusion and Future Scope	30
3.7. References	31

CHAPTER 4

PROJECT 2: OCCUPANCY BASED SMART SYSTEM

4.1. Abstract	32
4.2. Methodology	33
Phase 1: Hardware Setup and Sensor Configuration	
Phase 2: GPIO Verification and Logic Scripting	
Phase 3: Automation Logic	
Phase 4: Data Logging and Timestamping	
Phase 5: Deployment and Dashboard Visualization	
4.3. System Architecture	36
4.4. Hardware and Software Design	37
4.5. Results and Discussion	39

4.6. Conclusion and Future Scope	41
4.7. References	42

CHAPTER 5

PROJECT 3: SMART AIR MONITORING SYSTEM

5.1. Abstract	43
5.2. Methodology	44
Phase 1: Sensor Familiarization and Protocol Integration	
Phase 2: System Architecture and Sensor Interface	
Phase 3: Backend Communication Pipeline	
Phase 4: Frontend Dashboard Development	
Phase 5: Data Handling and Flow Control	
5.3. System Architecture	47
5.4. Hardware and Software Design	48
5.5. Results and Discussion	50
5.6. Conclusion and Future Scope	52
5.7. References	53

CHAPTER 6

CONCLUSION & FUTURE SCOPE

6.1. Summary of Internship Experience	54
6.2. Project Comparison & Integration Potential	55
6.3. Skills & Technical Competencies Gained	56
6.4. Future Scope and Industrial Relevance	58
6.5. Final Remarks	59

CHAPTER 7

REFERENCES	61
-------------------------	-----------

LIST OF ABBREVIATIONS	62
------------------------------------	-----------

LIST OF FIGURES	63
------------------------------	-----------

LIST OF TABLES	64
-----------------------------	-----------

APPENDIX	65
-----------------------	-----------

INTRODUCTION

1.1 IoT in Modern Applications

The Internet of Things (IoT) represents a paradigm shift in technology, where the physical world is seamlessly integrated with the digital realm. By enabling everyday objects and systems to **sense, collect, transmit, and respond to data**, IoT has transformed traditional processes into **intelligent, autonomous, and highly responsive systems**. From **smart homes** and **intelligent transportation systems** to **advanced healthcare** and **industrial automation**, IoT has become a **cornerstone of innovation** across diverse sectors.

Modern applications of IoT are built upon a combination of **embedded systems, cloud computing, data analytics, wireless communication, and machine learning**. These technologies converge to offer key benefits such as:

- **Real-time monitoring and decision-making**
- **Remote accessibility and control**
- **Operational automation**
- **Energy efficiency and cost savings**
- **Enhanced safety and user experience**

The accelerated adoption of IoT is not merely a trend but a **fundamental response** to the increasing demand for **intelligent systems** that **optimize resources and improve quality of life**.

1.2 Motivation and Scope of the Thesis

This thesis focuses on the practical implementation of IoT technologies in three crucial domains—**Home Appliance Control**, **Occupancy-Based Automation**, and **Air Quality Monitoring**. Each of these projects addresses **real-world challenges** and demonstrates how IoT can be leveraged to design **smart environments** that are **efficient**, **responsive**, and **sustainable**.

1.2.1 Smart Home Appliance Control System

With the growing need for convenience, energy conservation, and **user-centric design**, IoT-based **smart home systems** have become increasingly popular. This project showcases the development of a **smart appliance control interface** that allows users to control **electrical appliances** such as **lights**, **fans**, **air conditioners**, and **washing machines** through a **digital dashboard**. The system utilizes **microcontrollers**, **sensors**, **actuators**, and **wireless communication** (e.g., **Wi-Fi**, **MQTT**) to offer **real-time remote control** and **status feedback**. The interface is designed to be compatible with **kiosk screens** and **mobile platforms**, offering **flexibility in user interaction**.

1.2.2 Occupancy-Based Automation System

Traditional automation often overlooks real-time human presence, leading to **energy wastage** and **inefficiency**. This project introduces an **occupancy-based smart automation system** that dynamically controls **appliances** and **lighting** based on the detection of **human presence**. By integrating sensors like **PIR (Passive Infrared)**, **ultrasonic**, and **IR break-beam sensors**, the system **intelligently adjusts environmental settings** to reduce **power consumption** and improve **user comfort**. The implementation is ideal for **residential buildings**, **offices**, and **institutional facilities** where **automated energy management** is crucial.

1.2.3 Real-Time Smart Air Quality Monitoring System

Environmental awareness and health concerns have necessitated the development of systems that can continuously **monitor** and **analyze air quality parameters**. This project presents a **real-time air quality monitoring system** that employs sensors like **MQ135**, **PMS5003**, and **DHT22** to measure **CO₂**, **PM2.5**, **PM10**, **temperature**, **humidity**, and **AQI levels**. Using **Raspberry Pi** and integration with **Flask-SocketIO** and **Node-RED**, the data is displayed in a **live web dashboard** with **alerting capabilities** and **data logging**. The system can be deployed in **indoor environments**, **public spaces**, or **industrial areas** to assess **pollution levels** and maintain **healthy living conditions**.

1.3 Objective of the Thesis

The primary objective of this thesis is to:

- Demonstrate the **integration of IoT components** for **real-time sensing, actuation, and monitoring**.
- Develop three **scalable, user-friendly, and impactful IoT-based systems** targeting key domains.
- Highlight the **technical, functional, and user-experience** aspects of IoT applications.
- Provide a **comparative analysis** of **design approaches, sensor integration, communication protocols, and user interface strategies** used across the three projects.

LITERATURE REVIEW

In the rapidly advancing field of embedded systems and intelligent **automation**, the **Internet of Things (IoT)** has gained prominence as a **transformative technology**. As devices become increasingly **interconnected**, the demand for **smart, real-time, and energy-efficient systems** continues to grow. This chapter presents a comprehensive review of the **existing literature, tools, and technologies** that form the foundation for the three core projects undertaken in this thesis: **Smart Kiosk-Based Appliance Control, Occupancy Sensor Automation, and Real-Time Smart Air Monitoring System**.

2.1 Communication Protocols for IoT Systems

One of the critical enablers of IoT applications is lightweight and efficient **communication protocols**. Among these, **MQTT (Message Queuing Telemetry Transport)** has emerged as a **dominant protocol** due to its **low bandwidth requirements, minimal overhead, and publish-subscribe messaging architecture**.

Studies such as **Sharma et al. [1]** have demonstrated the **effectiveness of MQTT in smart home environments**, noting its **scalability and real-time responsiveness** in comparison to traditional **HTTP-based methods**. MQTT is particularly suitable for **IoT devices with limited processing**

power and network connectivity, making it ideal for use in **all three of the systems** discussed in this thesis.

Furthermore, **Node-RED**, a **flow-based development tool**, has been validated by developers and researchers as an **effective platform** for **prototyping** and **deploying MQTT-driven workflows**. As explored by **Kumar and Iyer [2]**, Node-RED **simplifies the orchestration** of **sensor data, actuator commands, and external services** through a **user-friendly visual interface**, significantly **reducing development time**.

2.2 Sensor-Based Automation Systems

Automation based on sensor fusion and environmental input is a growing research area, especially in domains focused on **energy efficiency** and **responsive control systems**. The **Occupancy Sensor Automation** project in this thesis draws inspiration from works such as **Li and Lin [3]**, who investigated the use of **motion sensors, infrared detectors, and temperature monitoring** for **real-time space utilization**. These studies show that **dynamic automation systems**, triggered by **environmental stimuli**, offer measurable **energy savings** and **improve user comfort** in **smart buildings**.

Additionally, recent research by **Andersson et al. [4]** confirms the **reliability of CO₂ concentration** as a **proxy for human presence** in **enclosed environments**. Their work shows a **strong correlation** between **rising CO₂ levels** and **increased occupancy**, which forms the basis of our **multi-sensor fusion approach** combining **PIR motion detection** with **air quality metrics** for **occupancy inference**.

2.3 Real-Time Environmental Monitoring

The third project, Smart Air Monitoring System, leverages sensor **technologies** to measure **air quality parameters** such as **particulate matter (PM2.5, PM10)**, **carbon dioxide (CO₂)**, **temperature**, and **humidity**. Widely used sensors like the **PMS5003** and **MQ135** have been validated in **open-source environmental monitoring platforms** and **urban health projects**. According to **Zhang et al. [5]**, these sensors provide a **low-cost yet reliable alternative** to **industrial air monitoring stations** when **calibrated correctly** and combined with **data smoothing techniques**.

Additionally, **real-time data presentation** is critical for **user interaction** and **decision-making**. The use of **WebSocket-based dashboards** for **streaming sensor data without refreshing** the page has been highlighted in **modern web development frameworks**. Research by **Gupta and Singh [6]** indicates that such **real-time interfaces** significantly **enhance user experience** and **data awareness** in applications ranging from **home automation** to **industrial safety**.

2.4 Relevance of Integrated IoT Projects

This thesis integrates insights from existing literature to build three practical and scalable IoT solutions:

- **Kiosk Smart System:** A touchscreen-based interface enabling intuitive appliance control using **MQTT** and **Node-RED** for real-time command execution.
- **Occupancy Sensor Automation:** An intelligent automation logic that adjusts appliance behavior based on motion and **CO₂** sensor inputs.
- **Smart Air Monitoring System:** A live dashboard displaying environmental quality data using **WebSocket** communication and widely adopted sensors.

By aligning **practical implementations** with **academically supported methodologies**, this work reinforces the **relevance and future potential** of IoT in **real-world contexts**. The **design and execution** of these systems offer **hands-on exposure** to **embedded IoT development**, **backend integration**, and **data visualization**, bridging **theoretical research** with **functional applications**.

KIOSK SMART SYSTEM

PROJECT 1

KIOSK SMART SYSTEM

3.1 Abstract

The KIOSK SMART SYSTEM is a robust and scalable Industrial IoT (**IIoT**)-based home automation solution designed during the course of an internship. This system facilitates seamless control and monitoring of multiple smart appliances—including air conditioners, washing machines, refrigerators, lights, and fans—through a **centralized 21-inch touchscreen kiosk interface**.

The solution is powered by a **Raspberry Pi 4** functioning as both the local server and controller, interfacing with user commands via a web-based UI. The backend logic is built using **Node-RED** and **MQTT protocol** (via the Mosquitto broker), allowing for real-time bidirectional communication between the kiosk interface and appliances. This project serves as a proof-of-concept for a **modular, responsive, and network-enabled smart environment**.

3.2 Methodology

A structured, multi-phase development strategy was adopted to ensure the modularity, stability, and scalability of the system:

Phase 1: Initialization & Environment Setup

- Installed **Raspberry Pi OS** (Debian-based) on Raspberry Pi 4.
- Configured the **Apache2 web server** for local hosting of the HTML/CSS/JS frontend.
- Set the **Chromium browser in kiosk mode** to auto-launch the interface on boot for a seamless user experience.

Phase 2: Frontend Development

- Developed **individual appliance control modules** using HTML, CSS, and JavaScript.
- Responsive UI layout compatible with 21-inch touchscreen.
- Custom functionality for each appliance:
 - **AC**: Temperature slider, fan speed control, mode switching.
 - **Washer**: Program select, timer-based control, pause/start.
 - **Fridge**: Mode toggle and temperature status.
 - **Lights/Fans**: Toggle switches with brightness/speed controls.

Phase 3: Backend Integration

- Deployed **Node-RED** as the visual backend orchestrator.
- Used **Mosquitto MQTT broker** for lightweight, real-time appliance control.
- Designed **RESTful APIs** for device state query and command publishing.
- Configured JSON message formats for consistent command-response parsing.

Phase 4: Hosting and Remote Access

- Hosted the interface locally on Apache.
- Enabled **public access via Ngrok tunneling** for remote demos.
- Employed **Postman** to test API reliability and validate command-response flow.

Phase 5: Final Testing and Validation

- Conducted extensive integration testing to ensure **real-time command execution** and **status feedback**.
- Performed **simulated and actual appliance trials** for reliability.
- Incorporated debounce logic and safety mechanisms in control toggles.

3.3 System Architecture

The Kiosk Smart System architecture involves:

- A **local embedded server (Raspberry Pi)** running backend services and frontend UI.
- Appliance commands triggered from the **touchscreen kiosk UI**.
- Communication with appliances using **MQTT publish/subscribe pattern**.
- Backend processing handled by **Node-RED flows** and device logic mapped via API topics.
- Tunneling the webpages and the backend flows through **ngrok links**.

System Block Diagram

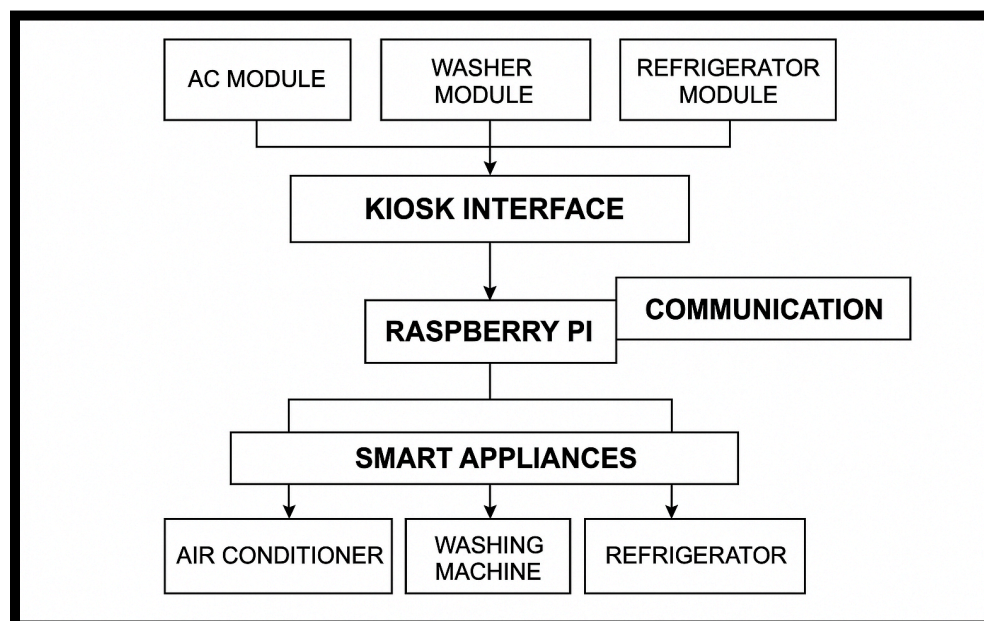


Fig 3.3.1 - Kiosk Smart System Block diagram

3.4 Hardware and Software Design

Hardware Components

COMPONENTS	DESCRIPTION
Raspberry Pi 4 (4GB RAM)	Local server, backend logic, frontend host

Table 3.4.1 - Hardware Components – Kiosk Smart System

Software Stack

LAYER	TECHNOLOGY USED
Operating System	Raspberry Pi OS (Debian-based)
Web Server	Apache2
Browser / Kiosk	Chromium (Kiosk Mode)
Frontend	HTML, CSS, JavaScript
Backend	Node.js, Node-RED
Communication	MQTT (via Mosquitto Broker)
Tunneling	Ngrok (for public demonstration)
Testing Tools	Postman (API testing and debugging)

Table 3.4.2 - Software Stack – Kiosk Smart System

3.5 Results and Discussion

The system was successfully implemented, tested, and demonstrated within a local environment. Key functionalities were validated across all modules, and MQTT-based communication ensured **low-latency control and feedback**. The following outcomes were observed:

System Highlights

- Intuitive, touchscreen-friendly **modular UI design**.
- **Real-time status feedback loop** integrated with MQTT topics.
- Smooth appliance toggling with **no perceptible delay**.
- **Remote access enabled** using secure tunnels via Ngrok.
- APIs validated and hardened using **Postman-based testing**.

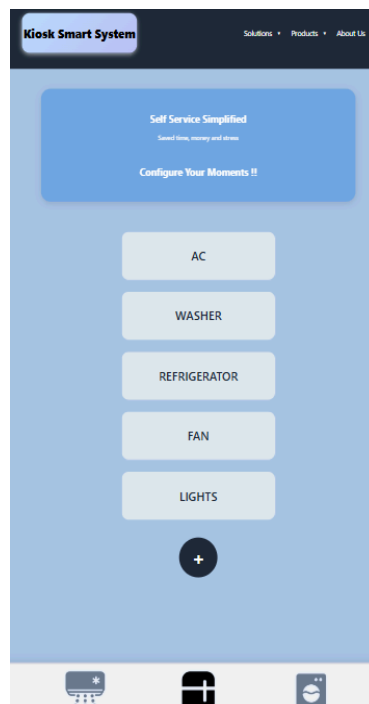


Fig 3.5.1 - Kiosk Smart System Webpage Preview

Appliance Control Modules

Air Conditioner Module

- Features: Power toggle, temperature slider, mode switch, fan speed control



Fig 3.5.2 - Kiosk Smart System Webpage Preview (AC Module)

Washing Machine Module

- Features: Program modes (Rinse, Spin Dry, Full Wash), timer, play/pause functionality.

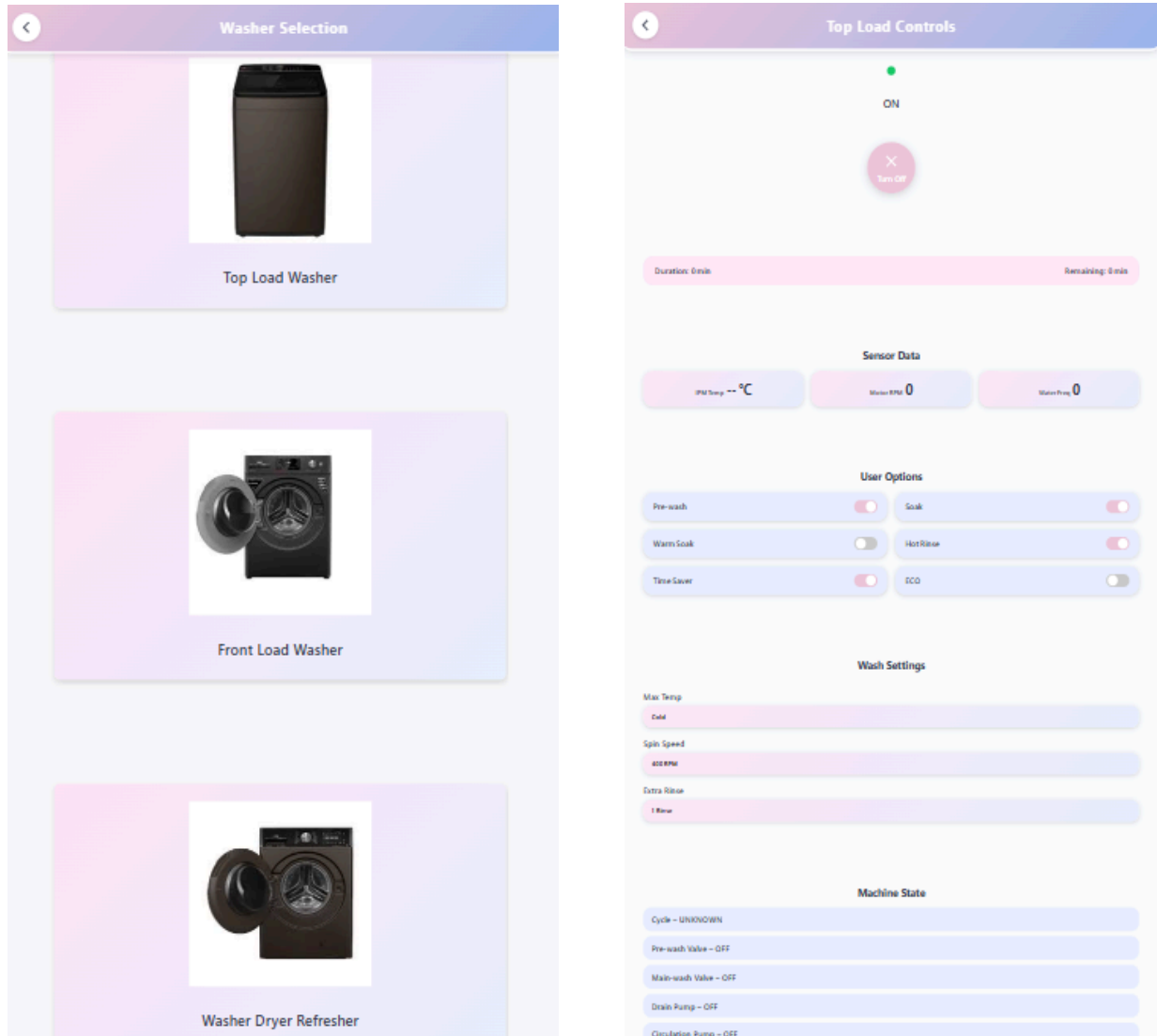


Fig 3.5.3 - Kiosk Smart System Webpage Preview(Washer Module)

Refrigerator Module

- Features: Power control, cooling mode selector, current temp display.



Fig 3.5.4 - Kiosk Smart System Webpage Preview (Refrigerator Module)

Lighting and Fan Module

- Features: Light on/off toggles, brightness control, fan speed adjustment

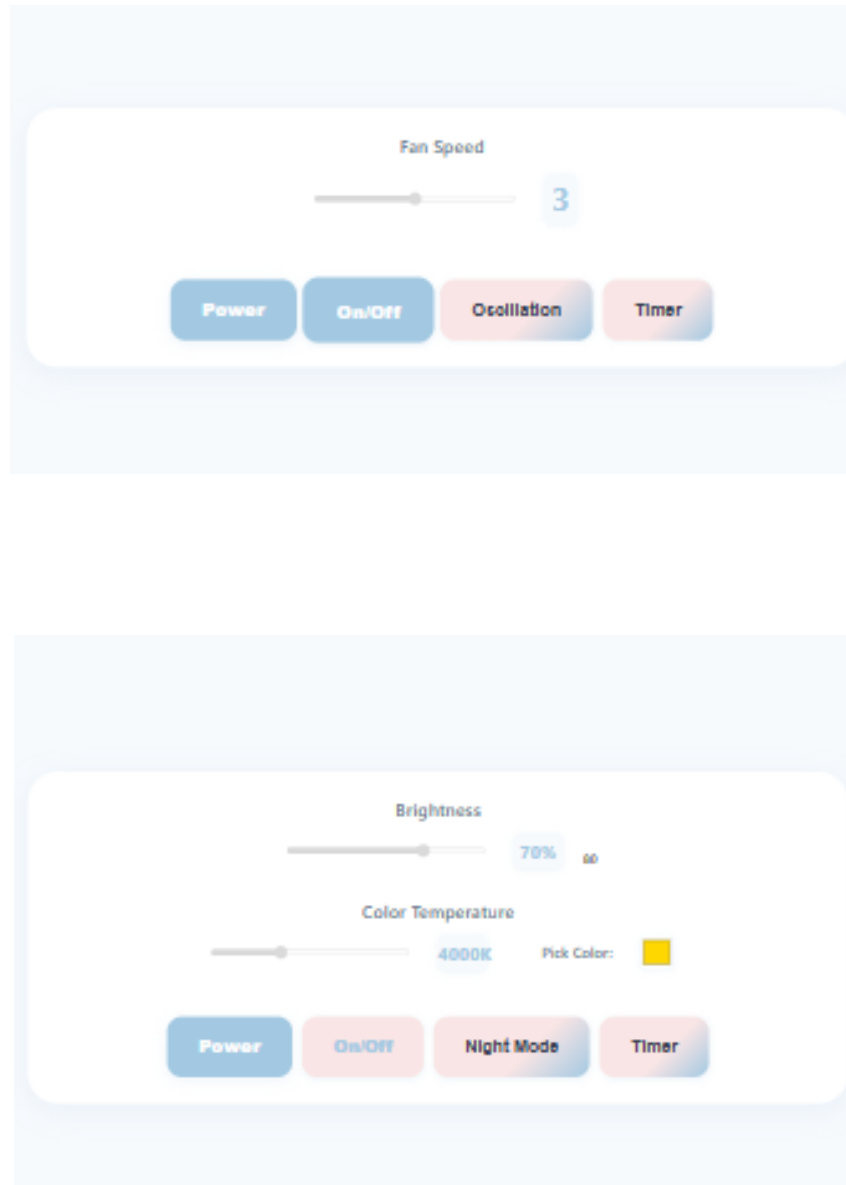


Fig 3.5.5 - Kiosk Smart System Webpage Preview (Light and Fan Module)

Node-Red Flow Diagram

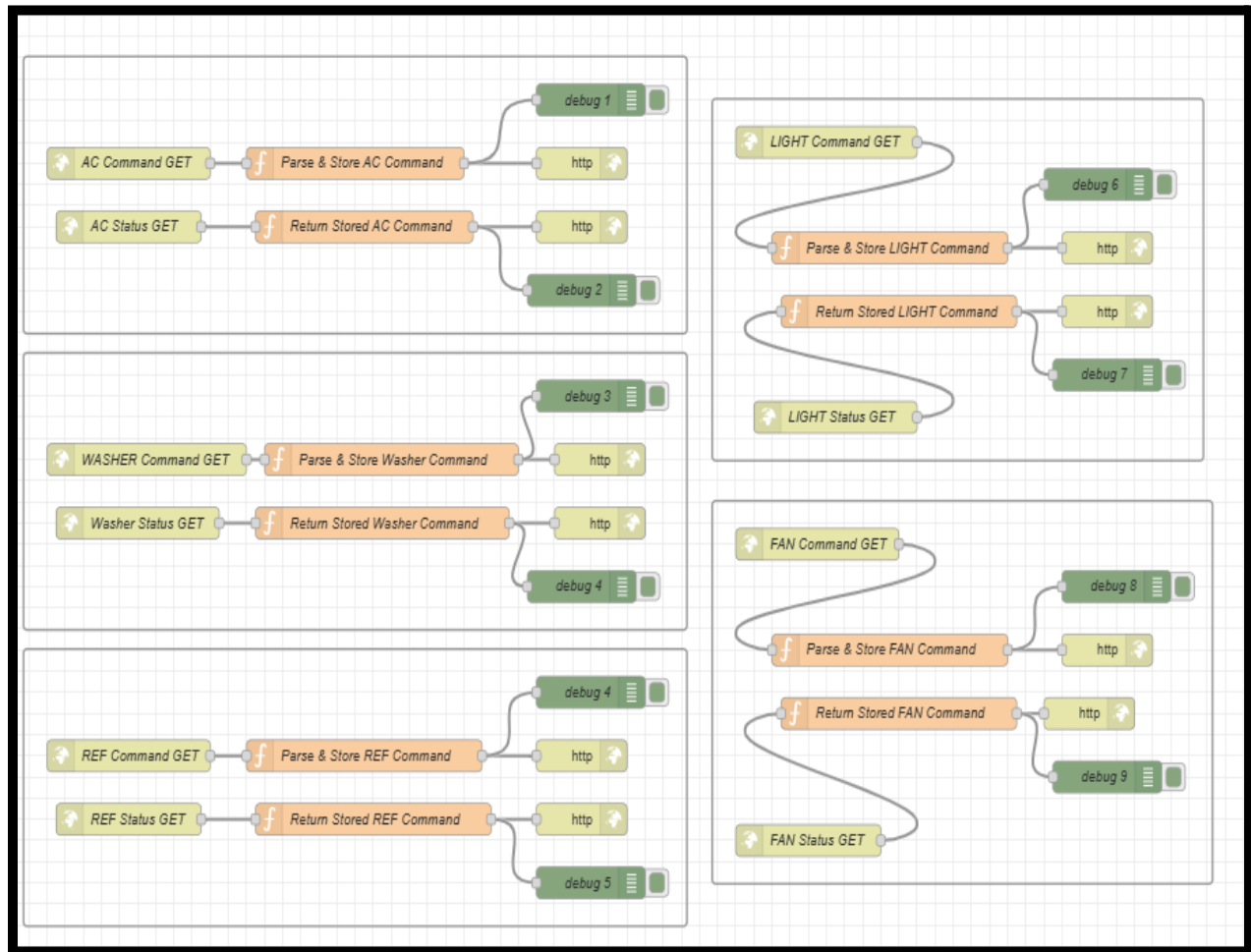


Fig 3.5.6 - Kiosk Smart System API'S

3.6 Conclusion and Future Scope

The KIOSK SMART SYSTEM successfully demonstrates how a **centralized, modular, and scalable** interface can serve as a control hub for smart home environments. Leveraging **open-source tools and real-time communication protocols**, the project reflects best practices in Industrial IoT implementation.

Future Enhancements:

- **Cloud Integration** using Firebase, AWS IoT Core, or Google Cloud for real-time analytics and data logging.
- **Voice Command Integration** using Google Assistant or Amazon Alexa SDKs.
- Migration to **component-based frontend frameworks** (React, Vue) for dynamic UI updates and improved state management.
- Implementation of **secure authentication layers** for personalized control and remote access security.
- Logging user interactions and appliance usage patterns for predictive maintenance or energy analytics.

3.7 References

1. Raspberry Pi Foundation. *Raspberry Pi Official Site*. <https://www.raspberrypi.org>
2. Mosquitto MQTT Broker. *Eclipse Foundation*. <https://mosquitto.org/>
3. Node-RED Documentation. <https://nodered.org/>
4. Mozilla Developer Network. *HTML, CSS, JS Docs*. <https://developer.mozilla.org/>
5. Ngrok: *Secure Tunnels to localhost*. <https://ngrok.com>
6. Postman API Platform. <https://www.postman.com/>
7. Apache HTTP Server Documentation. <https://httpd.apache.org/docs/>

OCCUPANCY BASED SMART SYSTEM

PROJECT 2

OCCUPANCY BASED SMART SYSTEM

4.1 Abstract

The Occupancy Sensor Automation System is a real-time, sensor-driven smart automation solution that intelligently manages appliance control based on human presence detection. Developed using a combination of **Raspberry Pi**, **Python scripting**, and **Node-RED**, the system integrates environmental and motion sensors to enable **energy-efficient automation** in enclosed environments such as smart homes, office cabins, or labs.

At its core, the system utilizes a **high-accuracy CO₂ sensor (SCD40)** via I²C protocol and a **PIR motion detector (HC-SR501)** interfaced through GPIO. A **2-channel relay module** controls appliances like lights or fans based on dynamically interpreted occupancy status. Additionally, the solution offers real-time visualization and historical data logging via the Node-RED dashboard, making it an ideal prototype for intelligent room automation.

4.2 Methodology

The project was developed in multiple phases, ensuring step-wise integration of sensor hardware, logic programming, and dashboard visualization:

Phase 1: Hardware Setup and Sensor Configuration

- The **SCD40 CO₂ sensor** was connected using the **I²C interface** and interfaced via the **Adafruit SCD4x Python library**.
- The **HC-SR501 PIR motion sensor** was connected to a Raspberry Pi GPIO pin and tested for human motion detection.
- A **2-channel relay module** was used to control connected appliances (e.g., light, fan), switching them ON/OFF based on logic derived from sensor readings.

Phase 2: GPIO Verification and Logic Scripting

- All GPIO pins were mapped and validated using Python scripts.
- **Custom Python programs** were written to read real-time sensor values and control GPIO outputs.
- The logic was embedded into **Node-RED using "exec" nodes**, allowing the backend to invoke Python scripts and process sensor data.

Phase 3: Automation Logic

- **Function nodes** in Node-RED were used to write logic for determining room occupancy.
- Occupancy status was based on:
 - **PIR motion** detection (immediate activity),
 - and **CO₂ concentration thresholds** (sustained human presence).
- Relay control was automated based on combined logic to minimize false positives and ensure responsive control.

Phase 4: Data Logging and Timestamping

- Sensor readings (CO₂ levels and motion state) were logged using Python's **CSV writer** module.
- Each event was timestamped and stored in a local **.csv** file for later analysis and performance evaluation.

Phase 5: Deployment and Dashboard Visualization

- A **Node-RED dashboard** was designed using **UI text, gauge, and chart nodes**.
- Real-time data for CO₂ ppm, motion state, and relay output were visualized.
- Final testing was performed in a closed room to validate automation reliability and system responsiveness.

4.3 System Architecture

The architecture is a hybrid of sensor-level interaction (hardware) and edge-level processing (Raspberry Pi). The block diagram below illustrates data flow from sensors to dashboard output via Raspberry Pi GPIO and Node-RED execution logic.

System Block Diagram

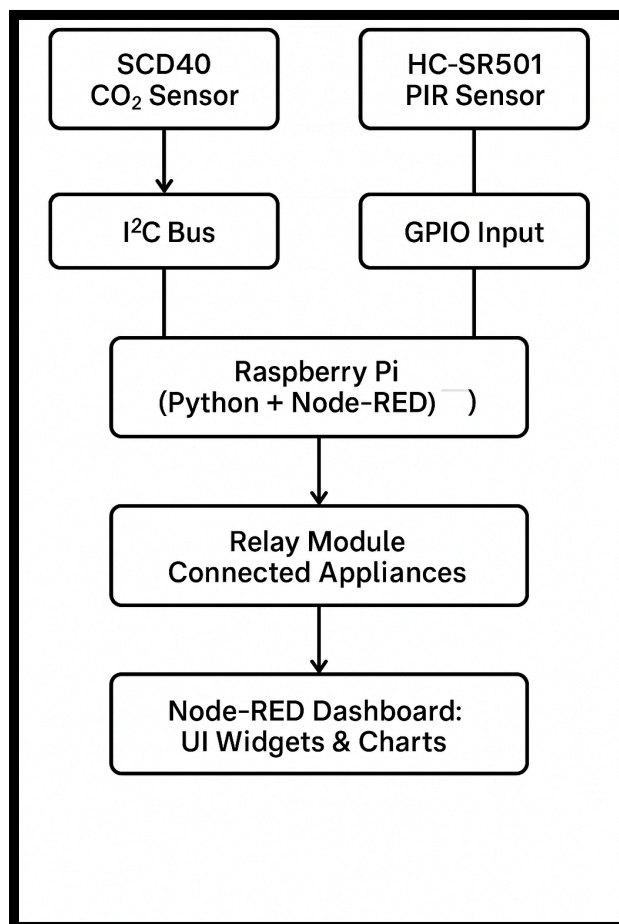


Fig 4.3.1 Occupancy Sensor System – Block Diagram

4.4 Hardware and Software Design

Hardware Components

COMPONENTS	DESCRIPTION
Raspberry Pi 4 Model B	Core controller with GPIO and I ² C interface
SCD40 CO ₂ Sensor	High-precision sensor for indoor air quality
HC-SR501 PIR Motion Sensor	Digital motion detector for presence sensing
MO24 2-Channel Relay Module	Switches for appliances (Light, Fan)
Breadboard & Jumpers	Circuit prototyping and connections

Table 4.4.1 - Hardware Components – Occupancy Smart System

Software Stack

LAYERS	TECHNOLOGY USED
Operating System	Raspberry Pi OS (Debian)
Programming	Python 3 (Adafruit SCD4x Library)
Automation Flow	Node-RED (Function, Exec, and Dashboard UI)
Communication	GPIO & I ² C
Data Logging	Python CSV Writer Module
Visualization	Node-RED Dashboard

Table 4.4.2 - Software Stack – Occupancy Smart System

GPIO Mapping Table

COMPONENTS	GPIO PINS USED	PIN FUNCTION
PIR Motion Sensor	GPIO17	Input
Relay Channel 1	GPIO23	Output
Relay Channel 2	GPIO24	Output
SCD40 (SDA)	GPIO2	I ² C Data
SCD40 (SCL)	GPIO3	I ² C Clock

Table 4.4.3 - GPIO Mapping – Occupancy Smart System

4.5 Results and Discussion

The system underwent successful implementation and testing in a controlled indoor environment. Key performance insights:

Performance Highlights

- **Occupancy detection accuracy** was high, particularly with the combination of motion and CO₂ sensing.
- **Relay toggling** for appliances responded within <1 second of input detection.
- **Sensor readings** were accurate and consistently logged with timestamps.
- The **Node-RED dashboard** effectively visualized real-time room occupancy and environmental parameters.
- **CSV logs** enabled tracking and post-analysis of usage patterns.

SAMPLE LOGGED DATA

TIMESTAMP	CO2	MOTION	ROOM STATUS
2025-06-20 10:01:03	412	YES	OCCUPIED
2025-06-20 10:05:44	403	NO	VACANT

Live Dashboard Preview

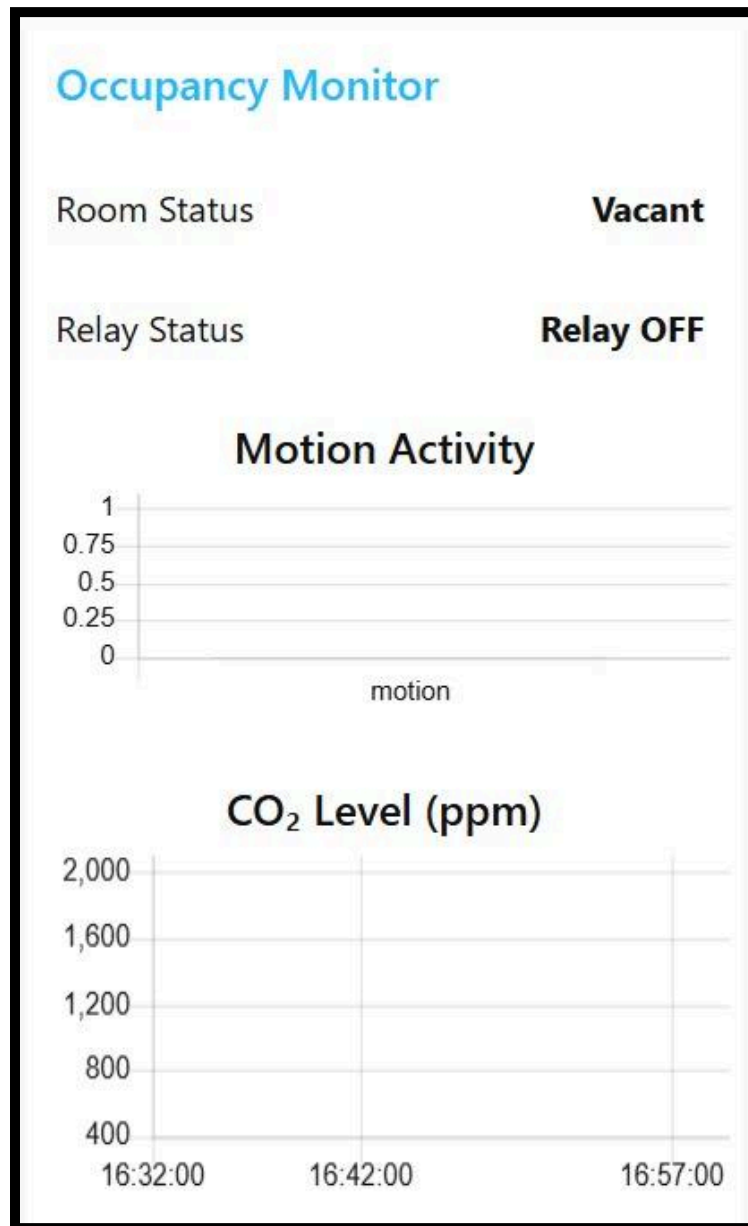


Fig 4.5.1 Live Node-RED Dashboard

Node-RED Flow Diagram

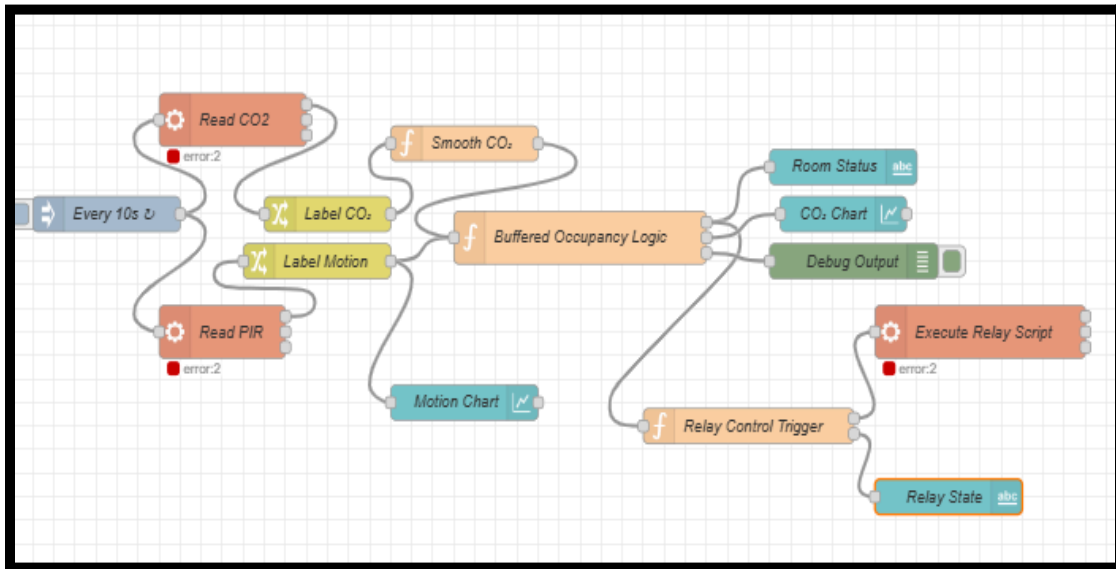


Fig 4.5.2 Live Node-RED Flow

4.6 Conclusion and Future Scope

This project demonstrates a practical and reliable approach to **automated occupancy-based appliance control** using low-cost sensors and open-source tools. By **combining environmental sensing with motion detection**, the system enhances convenience in smart environments.

Potential Enhancements

- **Integrate DHT22** sensor to log temperature and humidity.
- **Cloud Integration** with Firebase, Azure, or AWS IoT for remote monitoring and analytics.
- **Merge with the Kiosk Smart System** for unified smart home control.
- Enable **mobile notifications or alerts** in case of unauthorized access or abnormal CO₂ levels.

- Apply **machine learning models** to improve occupancy prediction and energy optimization over time.

4.7 References

1. Adafruit SCD40 Sensor Library Documentation – <https://learn.adafruit.com/scd40>
2. Raspberry Pi GPIO Documentation – <https://www.raspberrypi.org/documentation>
3. Node-RED Official Docs – <https://nodered.org/docs/>
4. PIR Motion Sensor (HC-SR501) Datasheet
5. MO24 Relay Module Datasheet
6. Python CSV Logging Tutorial – <https://realpython.com/python-csv/>

SMART AIR MONITORING SYSTEM

PROJECT 3

SMART AIR MONITORING SYSTEM

5.1 Abstract

The **Smart Air Monitoring System** is a real-time, IoT-driven environmental sensing platform designed to monitor indoor air quality with high precision. Implemented using a **Raspberry Pi** microcontroller and integrated with multiple air and climate sensors, the system captures key environmental parameters including **temperature, humidity, CO₂ concentration, PM2.5, PM10, and Air Quality Index (AQI)**.

The solution utilizes a hybrid communication protocol approach—**I²C, UART, GPIO, and ADC**—to interface with sensors. Real-time data is transmitted to a **Node-RED backend** using **MQTT** and subsequently streamed to a **custom HTML/CSS/JS frontend** using **WebSocket**. The platform supports **live updates, warning thresholds, and JSON data formatting**, establishing it as a compact, cost-effective monitoring solution for smart homes, offices, and indoor industrial environments.

5.2 Methodology

The project was developed in structured phases focusing on sensor calibration, communication reliability, and UI visualization:

Phase 1: Sensor Familiarization and Protocol Integration

- Studied the operating characteristics of:
 - **SCD40** for CO₂ sensing (I²C)
 - **DHT22** for temperature and humidity (GPIO)
 - **PMS5003** for particulate matter PM2.5/PM10 (UART)
 - **MQ135** for gas-based AQI estimation (ADC)
- Used **Adafruit CircuitPython** and **custom UART decoding** to ensure sensor compatibility.

Phase 2: System Architecture and Sensor Interface

- Connected all sensors to Raspberry Pi GPIO pins using appropriate communication protocols.
- Verified individual sensor outputs using standalone Python scripts.
- Standardized data formats across sensors to ensure JSON compatibility and consistency.

Phase 3: Backend Communication Pipeline

- Deployed **Mosquitto MQTT broker** to facilitate real-time communication between sensors and the local server.
- Used **Node-RED flows** to subscribe to MQTT topics, parse JSON payloads, and transmit data to the frontend via WebSocket.
- Added threshold logic to trigger alerts for elevated CO₂ and AQI values.

Phase 4: Frontend Dashboard Development

- Created a **real-time web interface** using HTML, CSS, and JavaScript.
- Integrated **WebSocket** to push live data without refreshing the page.
- Designed visual indicators (color codes, warning messages) to highlight poor air quality conditions.

Phase 5: Data Handling and Flow Control

- Created a **Python data aggregation script** that:
 - Polled sensors at set intervals.
 - Merged values into a standardized JSON format.
 - Published data to MQTT topics for downstream use.

5.3 System Architecture

The system's architecture captures sensor data using physical layer protocols, processes it on the Raspberry Pi, and communicates it via MQTT and WebSocket to a real-time dashboard. This architecture is modular, allowing easy expansion to additional zones or sensors.

System Block Diagram

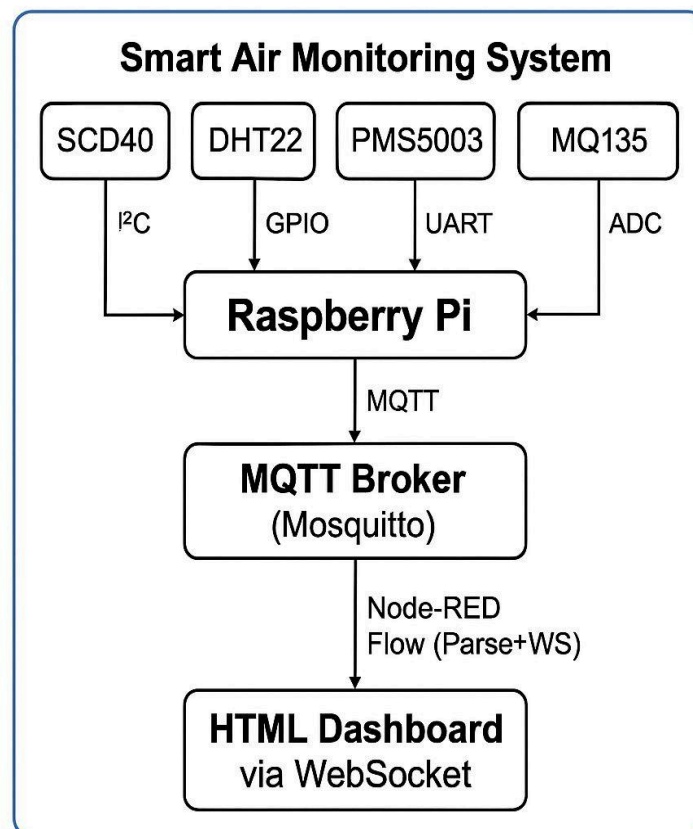


Fig 5.3.1 System Block Diagram - Smart Air Monitoring system

5.4 Hardware and Software Design

Hardware Components

COMPONENTS	INTERFACE USED	FUNCTIONALITY
Raspberry Pi 4B	All Protocols	Central controller and host
SCD40	I ² C	CO ₂ concentration sensing
DHT22	GPIO Digital	Temperature and humidity monitoring
PMS5003	UART (Serial)	Particulate matter sensing (PM2.5/PM10)
MQ135 + ADS1115	Analog via I ² C	AQI estimation via gas concentration
Breadboard & Wires	—————	Circuit prototyping

Table 5.4.1 Hardware components - Smart Air Monitoring System

Software Stack

COMPONENTS	TECHNOLOGY USED
OS	Raspberry Pi OS
Scripting	Python 3 with Adafruit & UART Libraries
Data Handling	JSON formatting & MQTT messaging
Server Layer	Mosquitto MQTT Broker, Node-RED
Frontend	HTML, CSS, JavaScript
Visualization	WebSocket real-time updates (Node-RED output)
Communication	UART (PMS5003), I ² C (SCD40, MQ135 ADC), GPIO

Table 5.4.2 Software Stack - Smart Air Monitoring System

Communication Overview

SENSORS	PROTOCOL	Pi INTERFACE
SCD40	I ² C	GPIO2 (SDA) GPIO3 (SCL)
DHT22	GPIO	GPIO4
PMS5003	UART	TX/RX via <code>/dev/serial0</code>
MQ135	ADC (I ² C)	ADS1115 via I ² C pins

Table 5.4.3 Communication Overview - Smart Air Monitoring System

5.5 Results and Discussion

The system was deployed and tested in an indoor setup, successfully capturing environmental metrics and reflecting them on the web interface in real time.

Key Observations

- All sensors were successfully interfaced and reported accurate, timely values.
- The dashboard achieved **sub-second update latency** using WebSocket streaming.

- MQTT-based data flow ensured **low bandwidth, high-efficiency transmission**.
- Custom thresholds for AQI and CO₂ were used to **trigger warning indicators**.
- Python scripts ensured **modular data aggregation** and sensor error handling.

Dashboard Preview

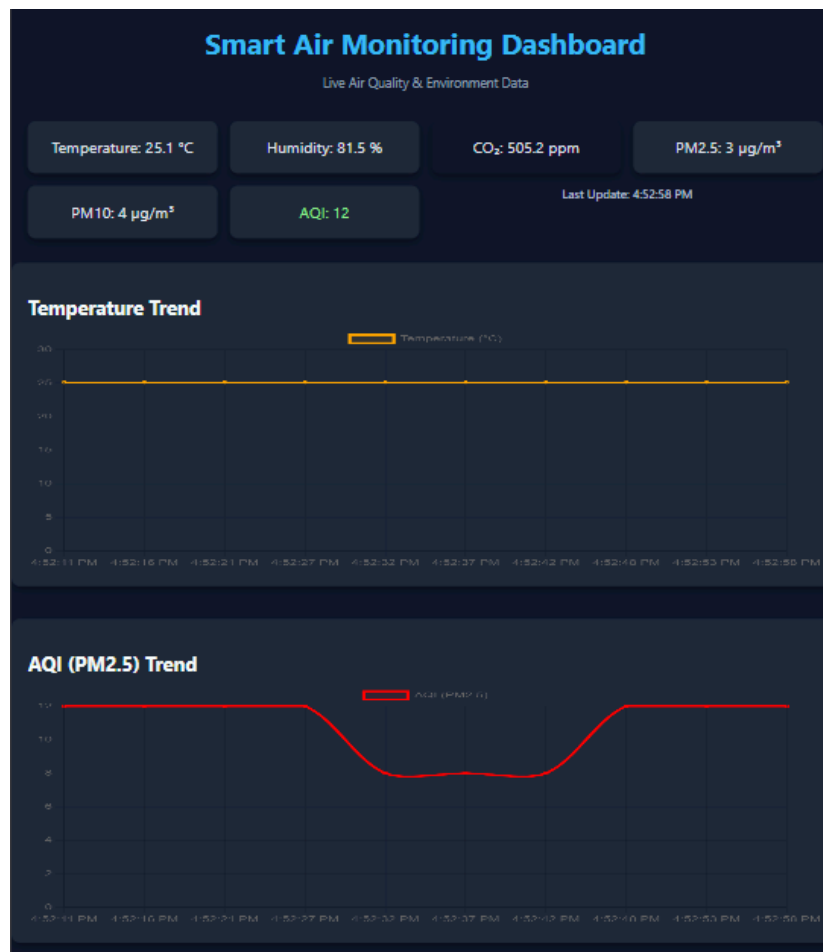
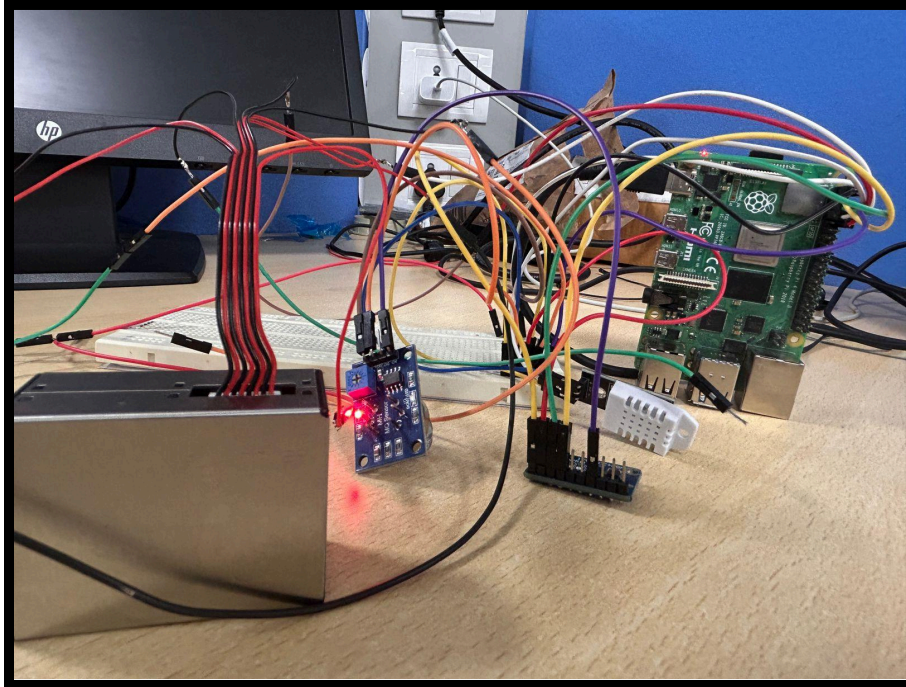


Fig 5.5.1 Dashboard Preview - Smart Air Monitoring System

Hardware Representation :



Node-RED Flow and VS code Terminal :

```
timestamp: "2025-06-  
26T16:48:08.263285"  
temperature_c: 24.8  
humidity_percent: 80.8  
co2_est_ppm: 505.2  
pm25_ugm3: 4  
pm10_ugm3: 4  
aqi_pm25: 17
```

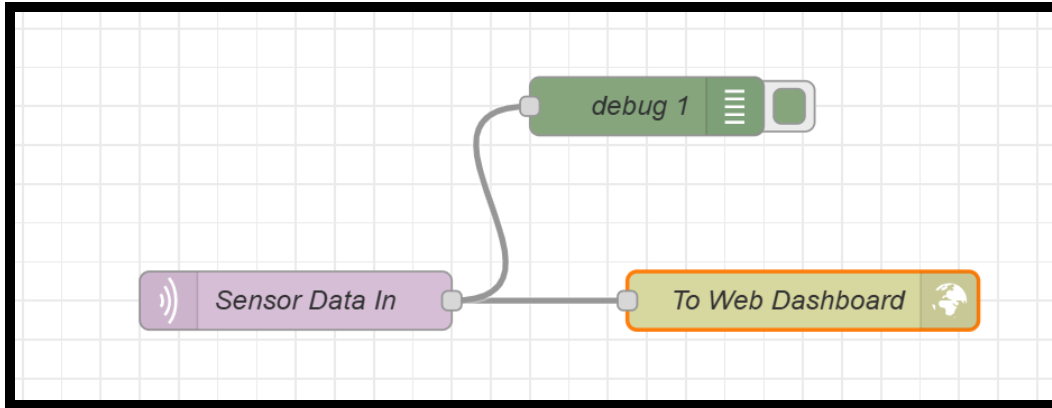


Fig 5.4.2 Live Nodered - Smart Air Monitoring System

```
pi@raspberrypi:~/Project $ python3 app_mqtt.py
/home/pi/Project/app_mqtt.py:20: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  mqtt_client = mqtt.Client()
INFO:root: 📡 Published: {'timestamp': '2025-06-26T16:47:20.877625', 'temperature_c': 24.7, 'humidity_percent': 81.3, 'co2_est_ppm': 524.1, 'pm25_ugm3': 3, 'pm10_ugm3': 4, 'aqi_pm25': 12}
INFO:root: 📡 Published: {'timestamp': '2025-06-26T16:47:26.142745', 'temperature_c': 24.7, 'humidity_percent': 81.1, 'co2_est_ppm': 530.5, 'pm25_ugm3': 3, 'pm10_ugm3': 4, 'aqi_pm25': 12}
```

5.6 Conclusion and Future Scope

The **Smart Air Monitoring System** successfully demonstrates the deployment of a **low-cost, real-time IoT platform** for air quality monitoring. By integrating open-source technologies, diverse sensor protocols, and real-time web visualizations, the system lays a scalable foundation for environmental intelligence in smart homes, institutions, and industry.

Future Enhancements

- **Real-time mobile alerts** for users during poor air quality conditions.
- **Historical data logging** using Firebase, MongoDB, or TimescaleDB.

- Integration of **GPS modules** for multi-location tracking and zone-based air maps.
- Implementation of **AI/ML models** for pollution level forecasting and health risk prediction.
- Add **voice assistant integration** for air quality query and suggestions.

5.7 References

1. Adafruit CircuitPython Sensor Libraries – <https://circuitpython.readthedocs.io>
2. PMS5003 Datasheet and Serial Communication Guide
3. Raspberry Pi UART & I²C Official Documentation – <https://www.raspberrypi.org/documentation>
4. Node-RED Official Documentation – <https://nodered.org/docs/>
5. WebSocket-MQTT Integration Guides – <https://www.hivemq.com/blog/mqtt-websockets>
6. EPA AQI Standards – <https://www.airnow.gov>

CONCLUSION

6.1 Summary of Internship Experience

This three-month industrial internship offered a comprehensive and hands-on exploration into the realm of **Internet of Things (IoT)**. Through the implementation of three distinct yet interrelated projects—**Kiosk Smart System**, **Occupancy Sensor Automation**, and **Smart Air Monitoring System**—the internship facilitated a deep understanding of IoT from sensor-level integration to cloud-agnostic communication and user-interface design.

Key experiential domains included:

- Sensor interfacing using **GPIO**, **I²C**, **UART**, and **ADC** protocols.
- End-to-end system development using **Node-RED**, **MQTT**, **WebSocket**, and **Python**.
- Dashboard/UI creation using **HTML**, **CSS**, **JavaScript**, and REST APIs.
- Real-time decision-making logic and energy-efficient automation strategies.

6.2 Project Comparison & Integration Potential

Feature	Kiosk Smart System	Occupancy Sensor System	Smart Air Monitoring System
Core Focus	Centralized Appliance Control	Motion-Driven Power Automation	Real-Time Air Quality Monitoring
Primary Sensors	GPIO Relays, UI Button Inputs	PIR (HC-SR501), CO ₂ Sensor (SCD40)	DHT22, PMS5003, MQ135, SCD40
Data Communication	MQTT, Node-RED, Apache Server	GPIO Logic, Node-RED Python Integration	MQTT + WebSocket + Node-RED
Frontend	Static Web Interface (HTML/CSS/JS)	Node-RED Dashboard	Live WebSocket Dashboard (HTML/JS)
Key Technologies	Apache, Node-RED, MQTT	Python, CSV Logging, Node-RED	MQTT, WebSocket, Python, JS Data Charts

Table 6.2.1 Project Comparison

Integrated System Potential

A fully unified **Smart Home IoT Framework** can be achieved by integrating all three systems under a **central Node-RED orchestrator**, where:

- The **Kiosk** acts as a centralized **command interface** for user-driven operations.
- The **Occupancy System** provides **autonomous control** logic to optimize energy consumption based on presence detection.
- The **Air Monitoring System** functions as an **environmental feedback layer**, triggering alerts and automation based on air quality metrics.

This layered model promotes modularity, scalability, and adaptability to real-world smart infrastructure needs

6.3 Skills & Technical Competencies Gained

Throughout the internship, a wide range of **technical proficiencies** and **interdisciplinary competencies** were developed:

Embedded Systems & IoT

- **Sensor interfacing** through GPIO, I²C, UART, ADC
- **Protocol handling** for real-time data acquisition

Software Development

- Backend scripting in **Python** for device communication and automation logic
- **MQTT messaging, WebSocket streaming, and Node-RED orchestration**
- API testing and validation using **Postman**

Web Technologies

- UI/UX design using **HTML, CSS, JavaScript**
- Real-time data binding and dashboard rendering
- RESTful communication and JSON parsing

Practical Applications

- Energy efficiency through occupancy-based control
- Indoor environmental monitoring for health and compliance
- Centralized control panel design for user convenience

6.4 Future Scope and Industrial Relevance

The systems developed during this internship are directly aligned with current trends in **smart infrastructure, environmental sustainability, and user-centric design**. Their **real-world applications** span across multiple industries:

Industrial & Societal Applications

- **Kiosk Systems** are ideal for deployment in **smart retail, hospitality, and automated service terminals**.
- **Occupancy Automation** finds relevance in **office buildings, educational institutions, and residential flats** for energy savings.
- **Air Monitoring Platforms** are critical in **healthcare facilities, industrial plants, and smart cities** for pollution control and wellness monitoring.

Future Expansion Possibilities

- **Cloud Connectivity** using **Firebase, AWS IoT, or Azure** for global data access.
- **Mobile App Development** for remote device control and analytics visualization
- **AI/ML Models** for:

- Predictive occupancy pattern recognition
 - Air quality forecasting and early anomaly detection
- **Security Integration** with facial recognition, voice commands, and user authentication
- Deployment of **multi-zone sensor grids** for large-scale smart buildings

6.5 Final Remarks

This internship served as a bridge between **academic knowledge** and **real-world industrial problem solving**. The experience cultivated a holistic understanding of full-stack IoT development—from sensor data acquisition to real-time dashboard visualization—and embedded critical skills needed in the growing fields of **smart automation**, **environmental intelligence**, and **embedded systems**.

It not only deepened my technical acumen but also instilled a **research-oriented mindset**—laying the groundwork for future contributions in advanced product development, smart system design, or postgraduate academic pursuits in IoT and intelligent systems.

REFERENCES

Each project has its own references; here's the combined list (shortened view):

1. Raspberry Pi Foundation – <https://www.raspberrypi.org>
2. Mosquitto MQTT Broker – <https://mosquitto.org>
3. Node-RED Documentation – <https://nodered.org/docs>
4. Mozilla MDN (HTML, CSS, JS) – <https://developer.mozilla.org>
5. Ngrok for Tunneling – <https://ngrok.com>
6. Postman API Testing – <https://www.postman.com>
7. Apache HTTP Docs – <https://httpd.apache.org/docs>
8. Adafruit SCD40 – <https://learn.adafruit.com/scd40>
9. Python CSV Logging – <https://realpython.com/python-csv>
10. CircuitPython Sensor Libraries – <https://circuitpython.readthedocs.io>
11. EPA AQI Standards – <https://www.airnow.gov>

LIST OF ABBREVIATIONS

Abbreviation	Full Form
IoT	Internet of Things
UI	User Interface
MQTT	Message Queuing Telemetry Transport
GPIO	General Purpose Input/Output
I ² C	Inter-Integrated Circuit
ADC	Analog to Digital Converter
PIR	Passive Infrared
AQI	Air Quality Index
PM2.5 / PM10	Particulate Matter (2.5µm / 10µm diameter)
REST API	Representational State Transfer Application Interface
CSV	Comma-Separated Values
SDK	Software Development Kit
IIoT	Industrial Internet of Things
JS	JavaScript

LIST OF FIGURES

Figure No	Title	Page No
Fig 3.3.1	Kiosk Smart System – Block Diagram	22
Fig 3.5.1	Webpage Preview	23
Fig 3.5.2	Webpage Preview (AC Module)	24
Fig 3.5.3	Webpage Preview (Washer Module)	25
Fig 3.5.4	Webpage Preview (Ref Module)	26
Fig 3.5.6	Webpage Preview (Light & Fan Module)	27
Fig 4.3.1	Occupancy Sensor System – Block Diagram	34
Fig 4.5.1	Live Node-RED Dashboard Preview	38

Fig 4.5.2	Live Node-RED Flow Preview	39
Fig 5.3.1	Smart Air Monitoring – Block Diagram	45
Fig 5.5.1	Webpage Preview-Smart Air Monitoring System	49
Fig 5.5.2	Node Red Flow - Smart Air Monitoring System	50

LIST OF TABLES

Table No	Title	Page No
Table 3.4.1	Hardware Components – Kiosk Smart System	22
Table 3.4.2	Software Stack – Kiosk System	22
Table 4.4.1	Hardware Components – Occupancy System	35
Table 4.4.2	Software Stack – Occupancy System	36
Table 4.4.3	GPIO Mapping Table – Occupancy System	36
Table 5.4.1	Hardware Components – Air Monitoring	46
Table 5.4.2	Software Stack – Air Monitoring	47
Table 5.4.3	Communication Overview – Sensor Protocols	48
Table 5.2.1	Project Comparison	54



THE END